

FEDERAL UNIVERSITY
OF SANTA CATARINA

EEL5105 – Circuitos e Técnicas Digitais
Aula 7

Prof. Héctor Pettenghi

hector@eel.ufsc.br
http://hectorpettenghi.paginas.ufsc.br

Material desenvolvido com apoio de arquivos de apresentação do livro de Frank Vahid

7. Projeto de Circuitos Sequenciais

7.1. Máquinas de Estados Finitos (FSM): Diagrama de estados

7.2. Projeto de FSM

7.1. Máquinas de Estados Finitos (FSM): Diagrama de estados

7.2. Projeto de FSM

7.1. Máquinas de Estados Finitos (FSM): Diagrama de estados

- Primeiro passo para o projeto de um **circuito combinacional**:
  - Montagem da **tabela verdade**
  - **Formalismo** que não envolve o tempo e, portanto, é insuficiente para lidar com circuitos sequenciais

7.1. Máquinas de Estados Finitos (FSM): Diagrama de estados

- Primeiro passo para o projeto de um **circuito combinacional**:
  - Montagem da **tabela verdade**
  - **Formalismo** que não envolve o tempo e, portanto, é insuficiente para lidar com circuitos sequenciais
- Solução para circuitos sequenciais: **máquina de estados finitos**
  - ***Finite State Machine*** (**FSM**)
  - Descreve o comportamento ao longo do tempo, a partir dos possíveis **estados** de um circuito
  - Indica o que pode causar **transições entre estados**
  - Indica como as **saídas do circuito** irão se comportar para cada estado.

7.1. Máquinas de Estados Finitos (FSM): Diagrama de estados

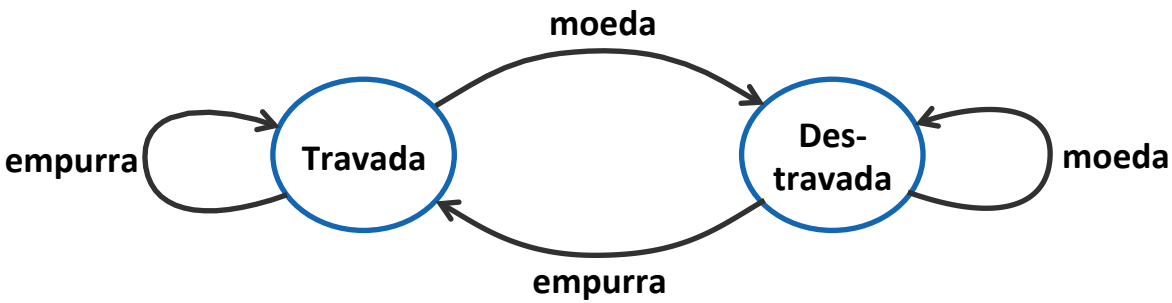
- **Exemplo 1**: **catraca** que abre com moedas



7.1. Máquinas de Estados Finitos (FSM): Diagrama de estados

- **Exemplo 1**: **catraca** que abre com moedas

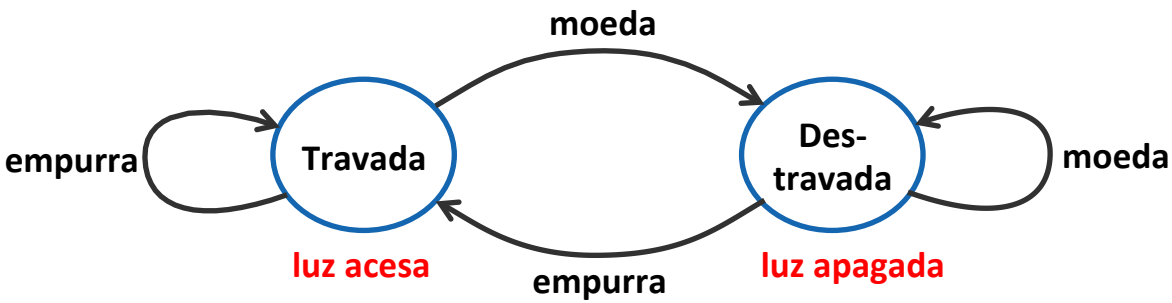




7.1. Máquinas de Estados Finitos (FSM): Diagrama de estados

- **Exemplo 1**: **catraca** que abre com moedas
  - Incluindo **indicador luminoso**:



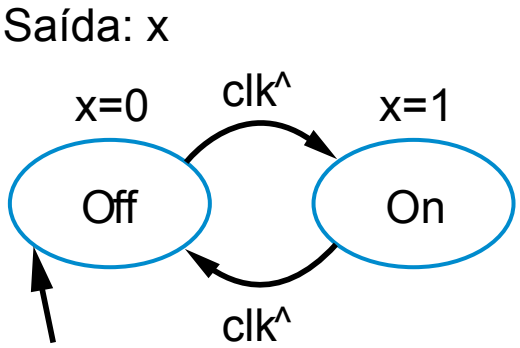


7.1. Máquinas de Estados Finitos (FSM): Diagrama de estados

- **Exemplo 2:** Circuito que tem uma saída **x** ligada a um **led**, sendo que tal **led** deve ficar piscando continuamente. Para isso, o **led** deve mudar de estado após cada transição positiva de *clock*.

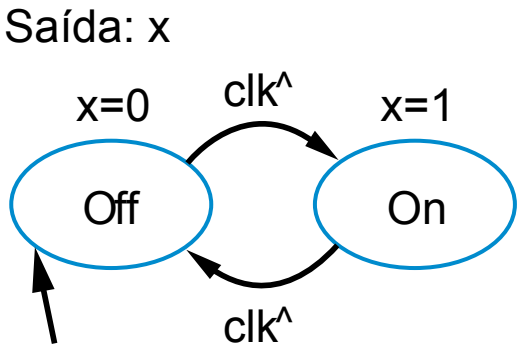
7.1. Máquinas de Estados Finitos (FSM): Diagrama de estados

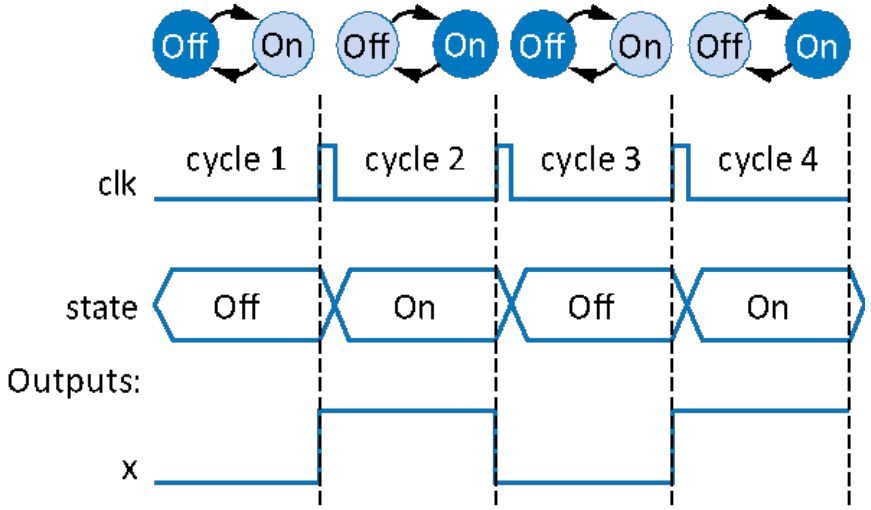
- **Exemplo 2:** Circuito que tem uma saída **x** ligada a um **led**, sendo que tal **led** deve ficar piscando continuamente. Para isso, o **led** deve mudar de estado após cada transição positiva de *clock*.
  - Estados: **On** e **Off**
  - **Diagrama de estados**:



7.1. Máquinas de Estados Finitos (FSM): Diagrama de estados

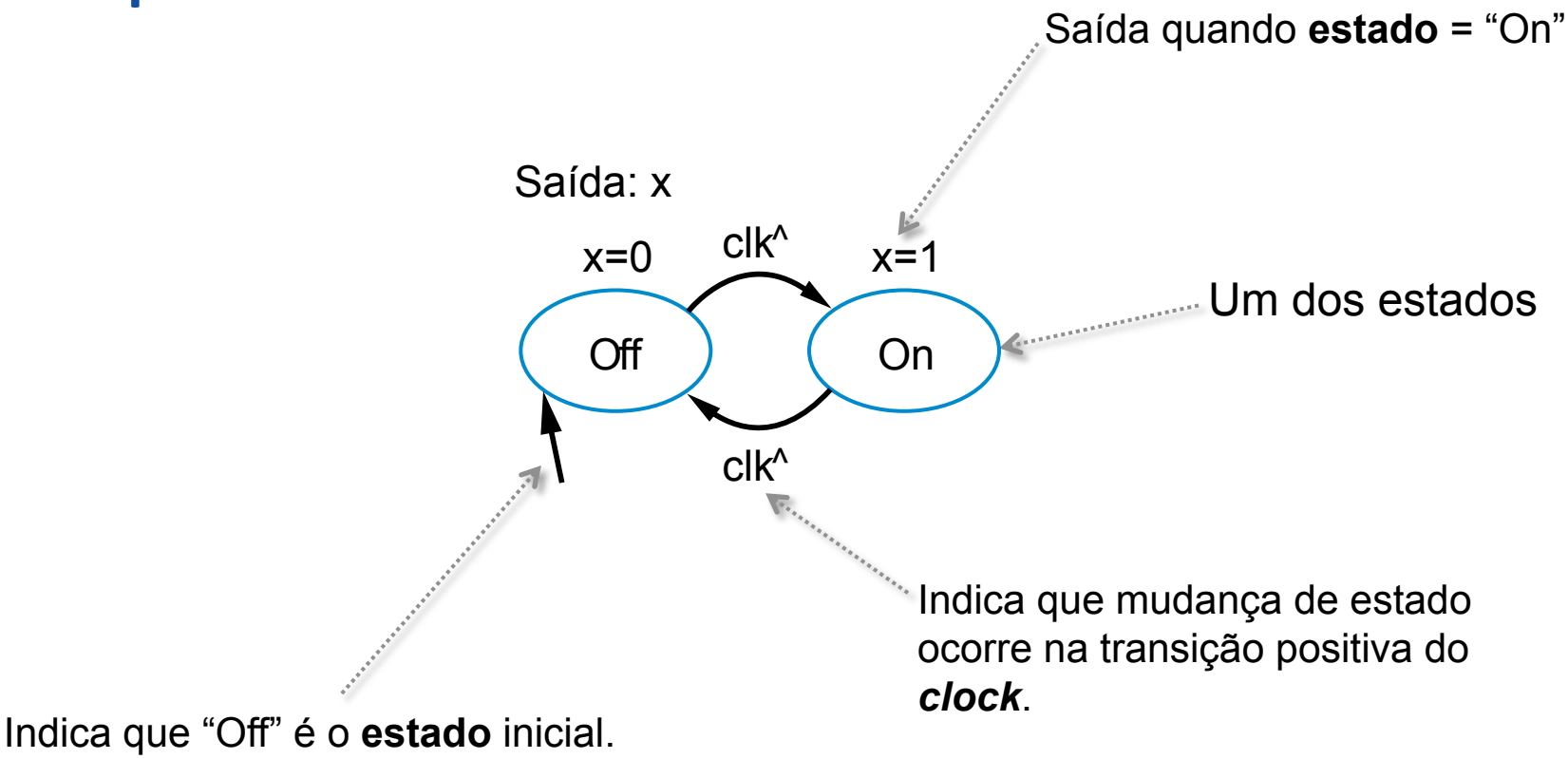
- **Exemplo 2:** Circuito que tem uma saída **x** ligada a um **led**, sendo que tal **led** deve ficar piscando continuamente.
  - Estados: **On** e **Off**
  - **Diagrama de estados** e **comportamento temporal**:





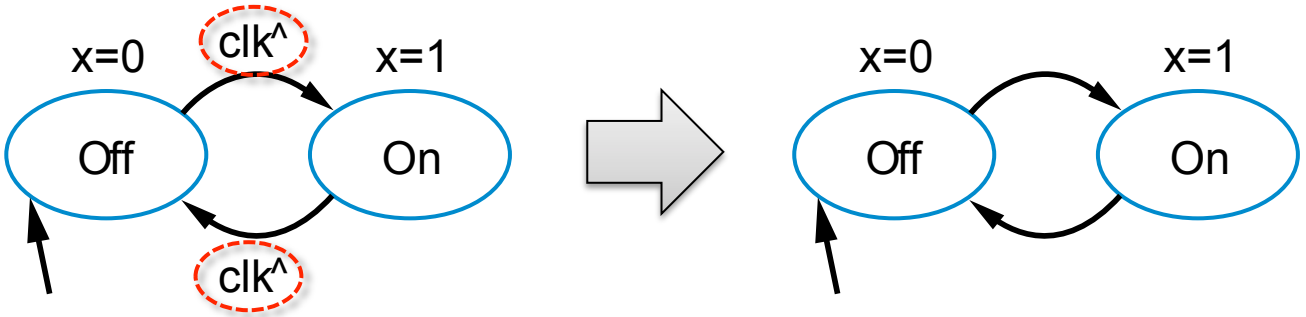
7.1. Máquinas de Estados Finitos (FSM): Diagrama de estados

- **Máquina de estados finitos**



7.1. Máquinas de Estados Finitos (FSM): Diagrama de estados

- Para **sistemas digitais**, é bastante comum a representação do *clock* de forma **implícita**, considerando que cada seta do diagrama já indica uma transição de *clock*:

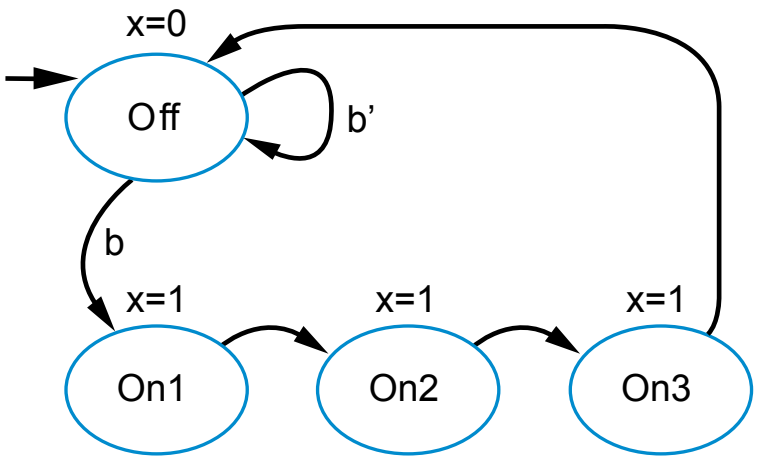


7.1. Máquinas de Estados Finitos (FSM): Diagrama de estados

- **Exemplo 3:** Circuito que mantém sua saída **x** em nível alto por 3 ciclos de *clock* quando um botão **b** for apertado.

7.1. Máquinas de Estados Finitos (FSM): Diagrama de estados

- **Exemplo 3:** Circuito que mantém sua saída **x** em nível alto por 3 ciclos de *clock* quando um botão **b** for apertado.
  - Entrada: **b**
  - Saída: **x**
  - Estados: **Off, On1, On2** e **On3**
  - Com *clock* **implícito**:

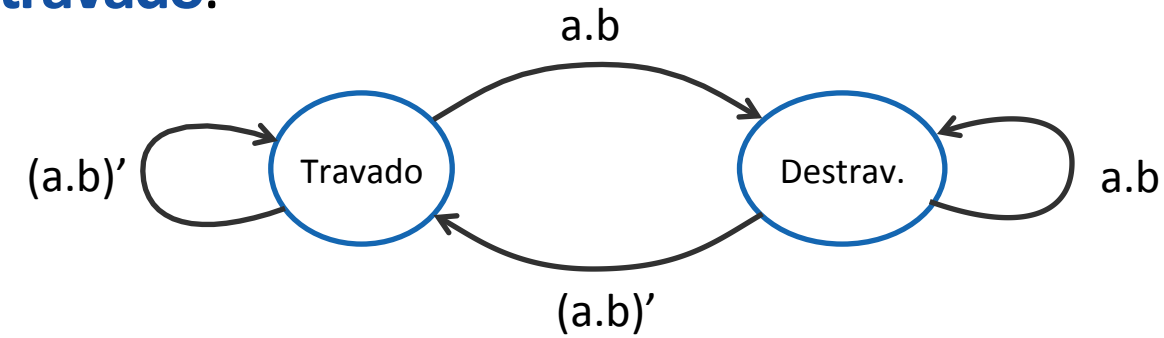


7.1. Máquinas de Estados Finitos (FSM): Diagrama de estados

- **Exemplo 4:** Faça o diagrama de estados com *clock* implícito de um sistema que fica em um estado chamado **travado** até que dois botões **a** e **b** sejam pressionados simultaneamente. Depois que tais botões forem apertados, o sistema deve ir para um estado **destravado** e permanecer nesse estado enquanto ambos os botões estiverem pressionados. Caso pelo menos um botão seja solto, o sistema deve voltar para o estado **travado**.

7.1. Máquinas de Estados Finitos (FSM): Diagrama de estados

- **Exemplo 4:** Faça o diagrama de estados com *clock* implícito de um sistema que fica em um estado chamado **travado** até que dois botões **a** e **b** sejam pressionados simultaneamente. Depois que tais botões forem apertados, o sistema deve ir para um estado **destravado** e permanecer nesse estado enquanto ambos os botões estiverem pressionados. Caso pelo menos um botão seja solto, o sistema deve voltar para o estado **travado**.



PROBLEMAS

- **Problema 7.1.** Faça o diagrama de estados de uma FSM para um circuito que tem um botão **b** ligado em sua entrada e deve deixar a sua saída **y** em nível alto por exatamente um período de *clock* após o aperto do botão. Assim, a saída deve ficar em nível alto por exatamente um ciclo de *clock* mesmo que o botão fique apertado por um tempo superior.

- **Problema 7.2.** Faça o diagrama de estados de uma FSM para um circuito que gere a saída ***Z*=1**, durante um ciclo de relógio quando na entrada ***X*** for "1" durante os três intervalos precedentes de relógio. Caso quatro ou mais ciclos de relógio ***X*** for "1", a saída será "0".

QUESTÃO 1: Tempo 5 min

7.1. Máquinas de Estados Finitos (FSM): Diagrama de estados.

7.2. Projeto FSM

7.2. Projeto FSM

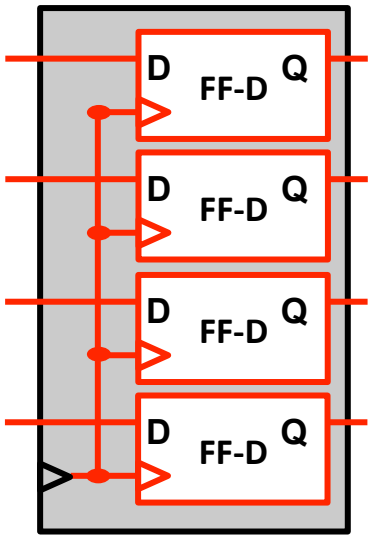
- Circuito sequencial com comportamento descrito por uma diagrama de estados.
- Duas componentes:
  - Bloco de lógica puramente **combinatória** para obter saidas e proximos estados a partir das entradas e estados atuais.
  - Elementos de memória (**registrador**), controlados por um sinal de relógio para armazenar os estados atuais.
- As máquinas de estado síncronas podem ser divididas em:
  - **Máquinas de Moore**: a saída depende *apenas* das variáveis de estado actuais;
  - **Máquinas de Mealy**: a saída é função das variáveis de estado actuais *e* do valor das entradas presentes no circuito

7.2. Projeto FSM

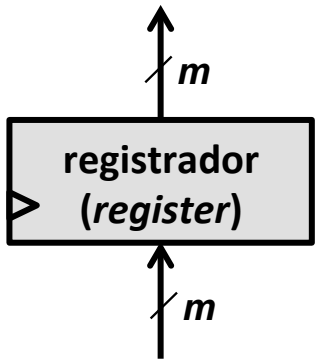
- Duas componentes:
  - Bloco de lógica puramente **combinatória** para obter saídas e próximos estados a partir das entradas e estados atuais.

Seguir os passos de projeto de circuitos combinatorios (tabelas de verdade)

  - Elementos de memória (**registrador**), controlados por um sinal de relógio para armazenar os estados atuais.

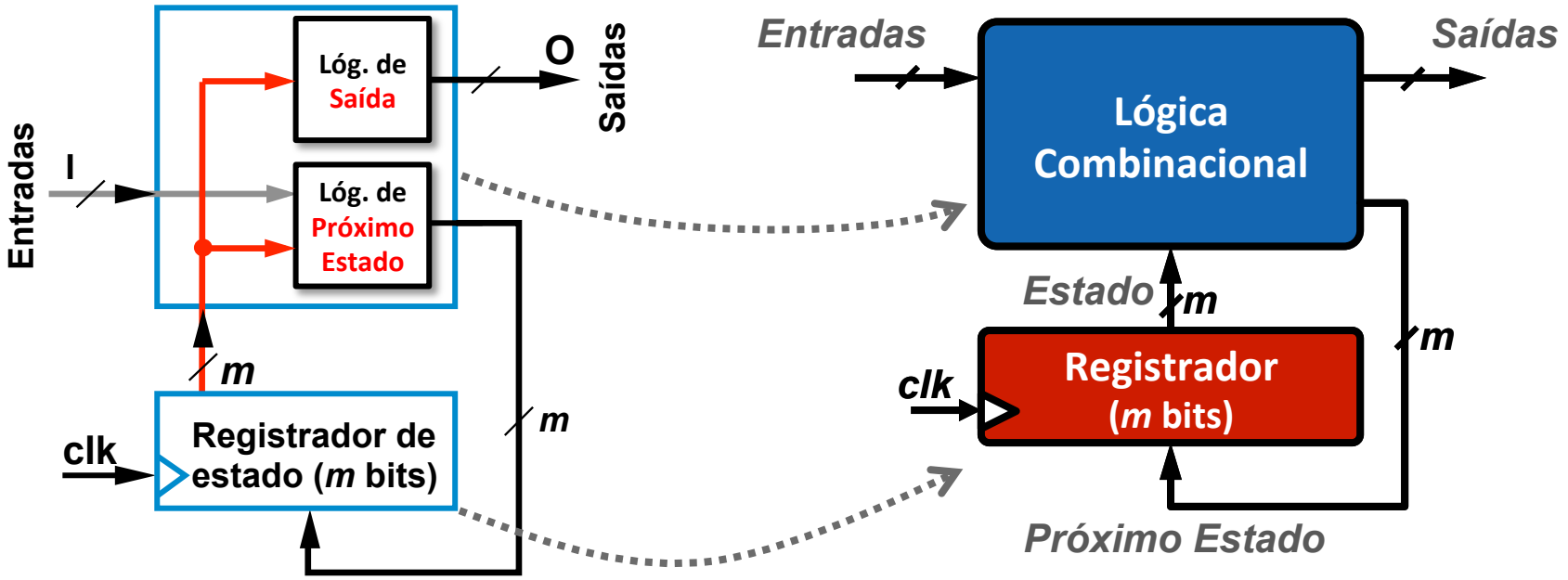


Exemplo de registrador com *m = 4* bits:



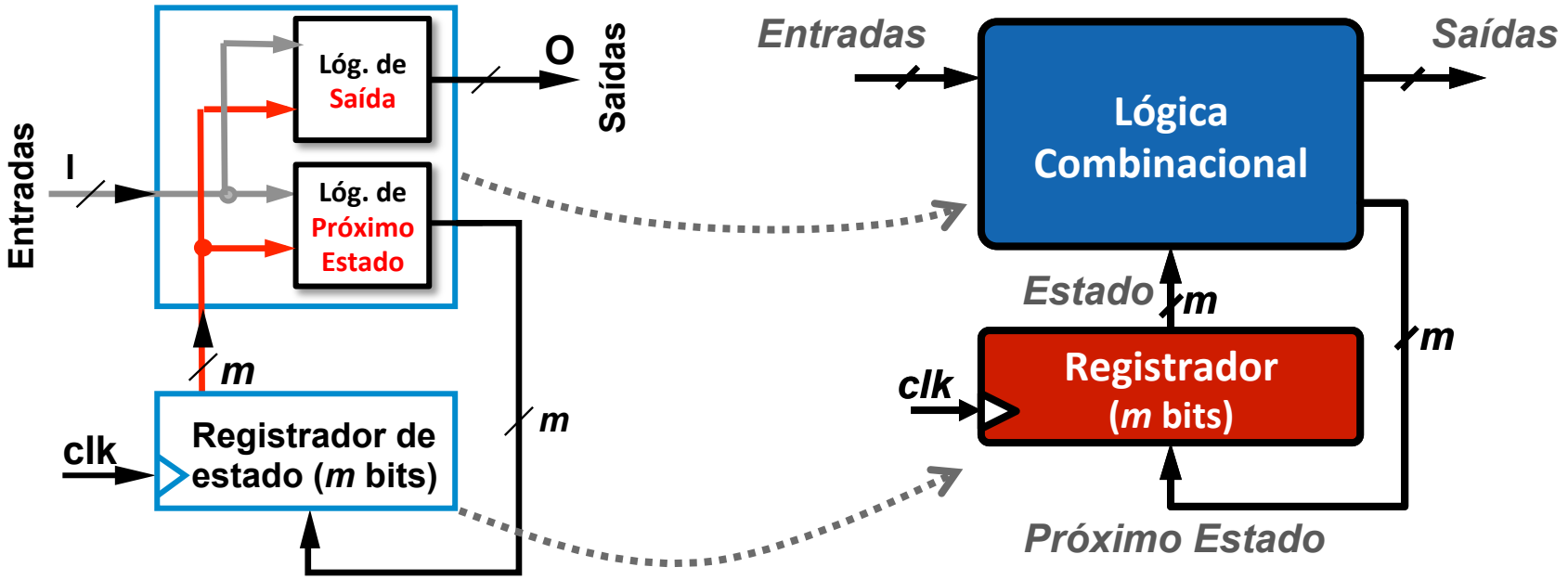
7.2. Projeto FSM

- Duas componentes:
  - Bloco de lógica puramente **combinatória** para obter saidas e proximos estados a partir das entradas e estados atuais.
  - Elementos de memória (**registrador**), controlados por um sinal de relógio para armazenar os estados atuais.
- **Modelo de Moore:**



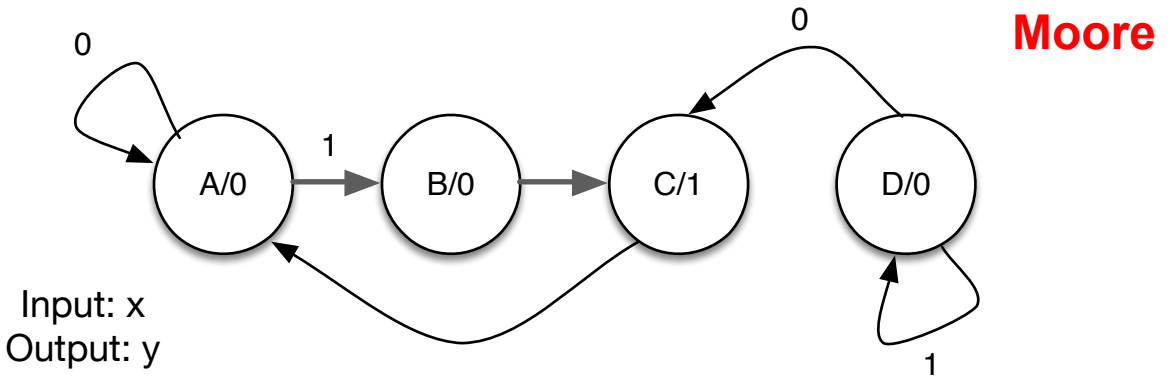
7.2. Projeto FSM

- Duas componentes:
  - Bloco de lógica puramente **combinatória** para obter saidas e proximos estados a partir das entradas e estados atuais.
  - Elementos de memória (**registrador**), controlados por um sinal de relógio para armazenar os estados atuais.
- **Modelo de Mealy:**

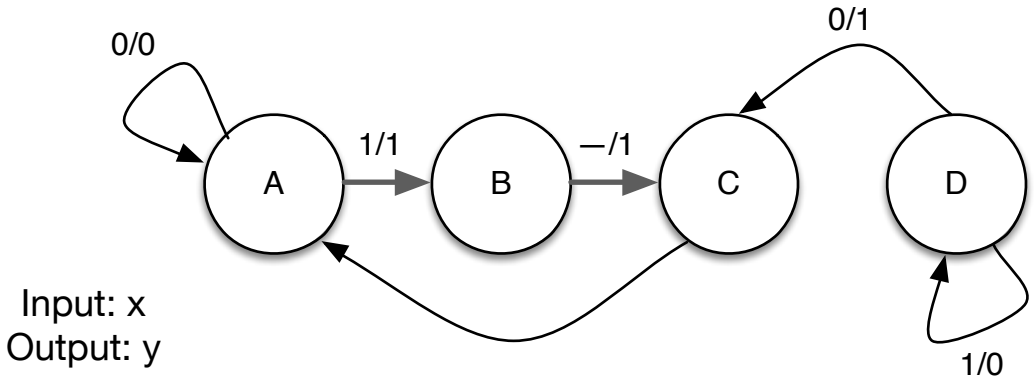


7.2. Projeto FSM

- **Exemplos de diagrama de estados:**



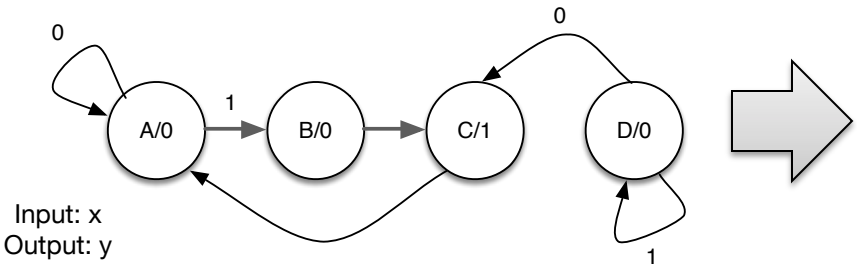
Moore

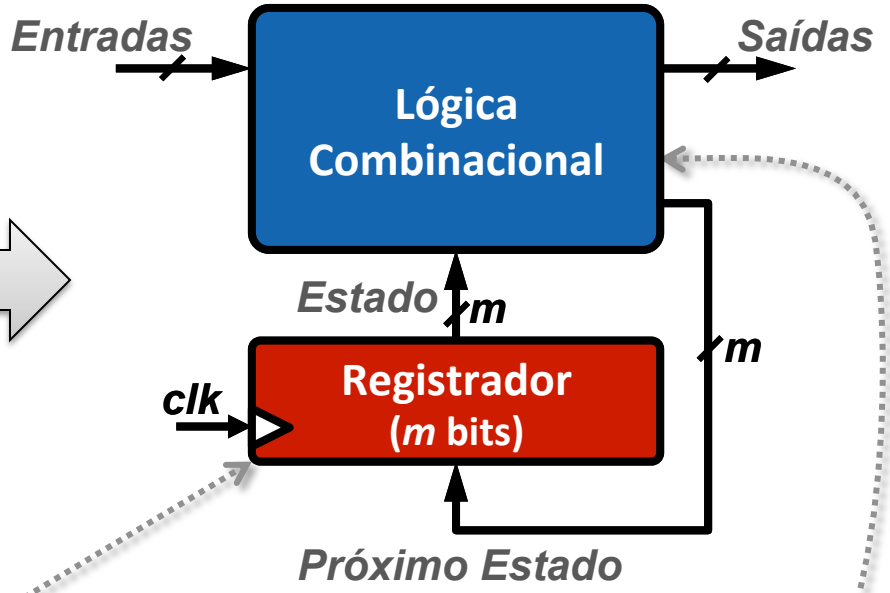


Mealy

7.2. Projeto FSM

- **Questão**: como transformar um diagrama de estados em um circuito lógico sequencial?





- **Para tal, basta**:
  - Codificar os **estados**
  - Escolher o **registrador** apropriado
  - Projetar a **lógica combinacional**

7.2. Projeto FSM

- Projecto de Circuitos Sequenciais Síncronos (FSM)
  - **Procedimento:**
    - **Especificação formal:**
      - Diagrama de estados
      - Fluxograma
    - **Projecto:**
      1. Codificação dos estados
      2. Tabelas de transição de estados
      3. Determinação das funções lógicas de saída e estado seguinte

7.2. Projeto FSM

- Projecto de Circuitos Sequenciais Síncronos
  - **Procedimento:**
    - **Especificação formal:**
      - Diagrama de estados ← **já visto nos exemplos anteriores…**
      - Fluxograma
    - **Projecto:**
      1. Codificação dos estados
      2. Tabelas de transição de estados
      3. Determinação das funções lógicas de saída e estado seguinte

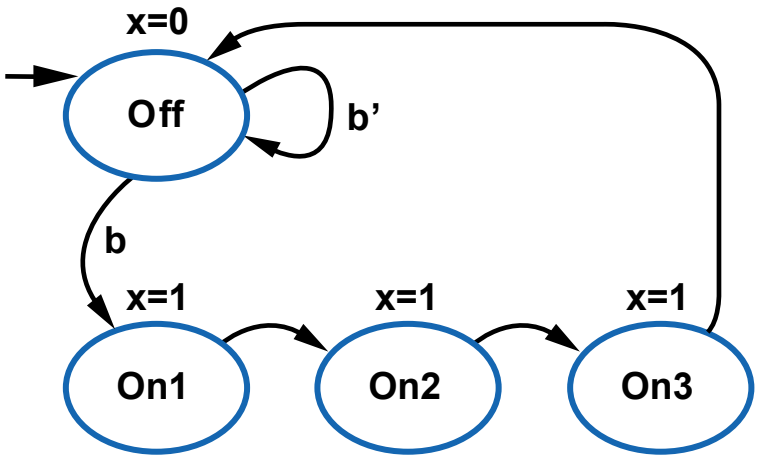
7.2. Projeto FSM

- **Exemplo:** circuito que mantém sua saída **x** em nível alto por 3 ciclos de clock quando um botão **b** for apertado.
    - Entrada: **b**
    - Saída: **x**
    - Estados:
      Off, On1, On2 e **On3**.
- **Especificação formal:** ***Obter o diagrama de estados***

7.2. Projeto FSM

- **Exemplo:** Circuito que mantém sua saída **x** em nível alto por 3 ciclos de clock quando um botão **b** for apertado.
  - Entrada: **b**
  - Saída: **x**
  - Estados:
    Off, On1, On2 e **On3**.
- **Especificação formal:** ***Obter o diagrama de estados***

Maquina Moore



Entrada: b; Saída: x

7.2. Projeto FSM

- Projecto de Circuitos Sequenciais Síncronos
  - **Procedimento:**
    - **Especificação formal:**
      - Diagrama de estados
      - Fluxograma
    - **Projecto:**
      1. Codificação dos estados
      2. Tabelas de transição de estados
      3. Determinação das funções lógicas de saída e estado seguinte

7.2. Projeto FSM

Codificação dos estados

- **Método *binário***: codificação usando o código binário
  - Método mais eficiente, pois usa o menor número de Flip-Flops possível
  - Menor número de funções para definir o próximo estado → menos portas lógicas (em geral…)
  - Considerando a existência de n estados (E_0, E_1, E_2,…, E_{n-1}) , a codificação usando código binário natural irá usar k Flip-Flops, em que k é o menor inteiro igual ou superior a $\log_2(n)$
    - Exemplo:
      - 6 estados (E_0, E_1, E_2, E_3, E_4, E_5)
      - $k = \lceil \log_2(6) \rceil = \lceil 2.584 \rceil = 3$ Flip-Flops

Estado	Codificação		
	S_2	S_1	S_0
E_0	0	0	0
E_1	0	0	1
E_2	0	1	0
E_3	0	1	1
E_4	1	0	0
E_5	1	0	1

Existem várias codificações possíveis!!!

7.2. Projeto FSM

Codificação dos estados

- **Método *One-hot***: codificação usando um Flip-Flop por estado
  - Usa tantos Flip-Flops quanto o número de estados

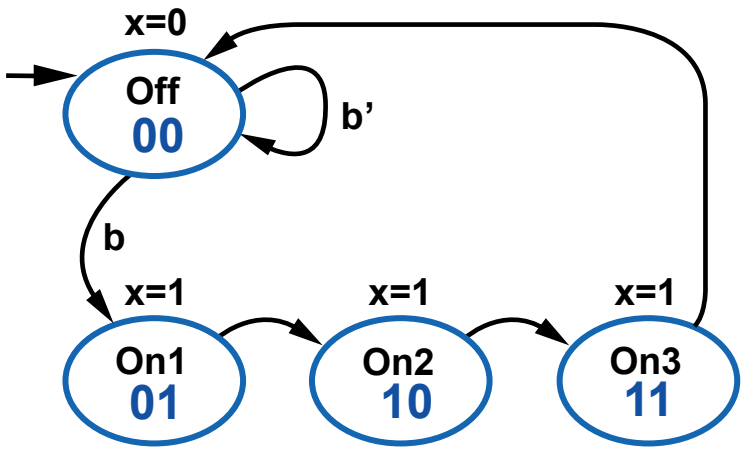
    → maior número de funções combinatórias a sintetizar!

    → mas cada uma destas funções é, em geral, mais simples…
  - Apenas um Flip-Flop tem a saída a '1' em cada instante
    - Exemplo:
      - 6 estados (E_0, E_1, E_2, E_3, E_4, E_5)
      - 6 Flip-Flops

Estado	Codificação					
	s_5	s_4	s_3	s_2	s_1	s_0
E_0	0	0	0	0	0	**1**
E_1	0	0	0	0	**1**	0
E_2	0	0	0	**1**	0	0
E_3	0	0	**1**	0	0	0
E_4	0	**1**	0	0	0	0
E_5	**1**	0	0	0	0	0

7.2. Projeto FSM

- **Passo 1:** ***Codificação os estados***
  - Padrão:



Codificação binária

Estado	Código
Off	00
On1	01
On2	10
On3	11

  - Qualquer codificação pode ser usada, desde que o código seja único para cada estado.

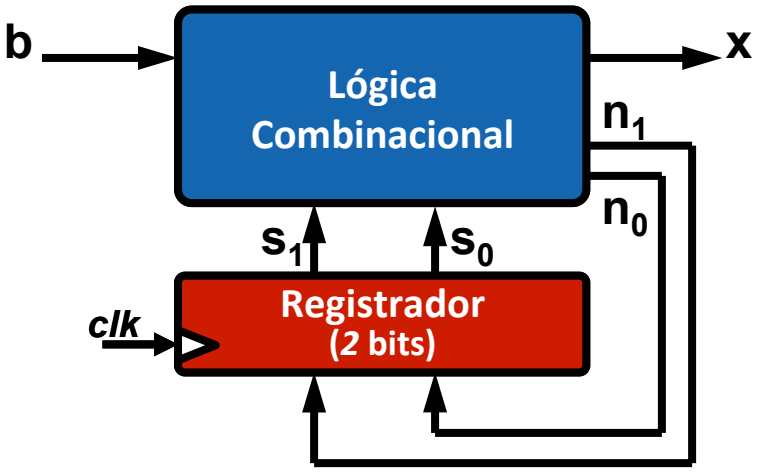
7.2. Projeto FSM

- Projecto de Circuitos Sequenciais Síncronos
  - **Procedimento:**
    - **Especificação formal:**
      - Diagrama de estados
      - Fluxograma
    - **Projecto:**
      1. Codificação dos estados
      2. Tabelas de transição de estados
      3. Determinação das funções lógicas de saída e estado seguinte

7.2. Projeto FSM

- **Passo 2: Tabelas de transição de estados**
  - Tabela verdade para a **parte combinacional**

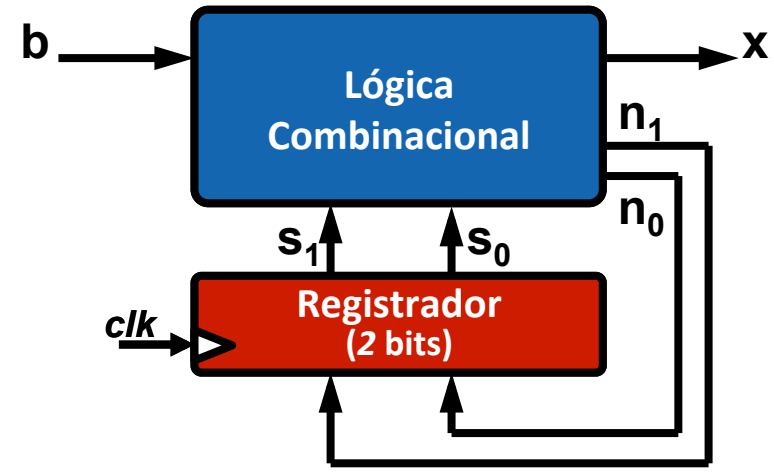
Entradas			Saídas		

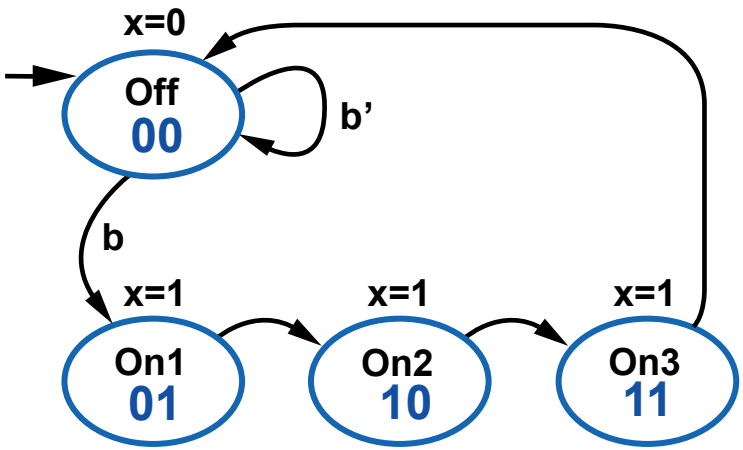


7.2. Projeto FSM

- **Passo 2:** Tabelas de transição de estados

Entradas			Saídas		
s_1	s_0	b	x	n_1	n_0
0	0	0			
0	0	1			
0	1	0			
0	1	1			
1	0	0			
1	0	1			
1	1	0			
1	1	1			

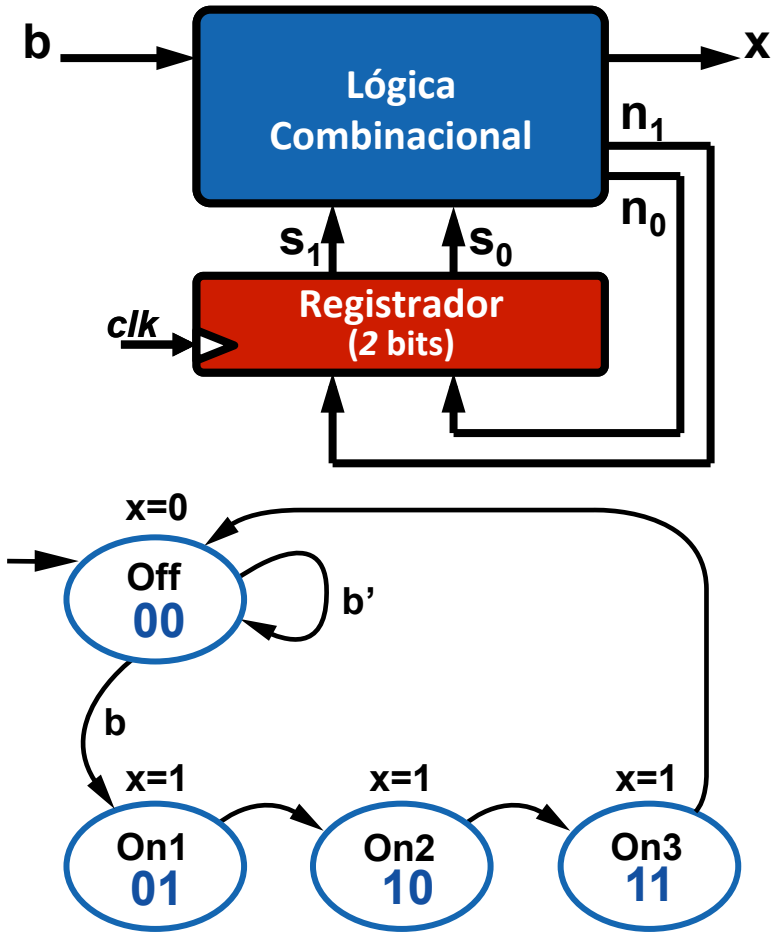




7.2. Projeto FSM

- **Passo 2:** Tabelas de transição de estados

Entradas			Saídas		
s_1	s_0	b	x	n_1	n_0
0	0	0	0	0	0
0	0	1	0	0	1
0	1	0	1	1	0
0	1	1	1	1	0
1	0	0	1	1	1
1	0	1	1	1	1
1	1	0	1	0	0
1	1	1	1	0	0



7.2. Projeto FSM

- Projecto de Circuitos Sequenciais Síncronos
  - **Procedimento:**
    - **Especificação formal:**
      - Diagrama de estados
      - Fluxograma
    - **Projecto:**
      1. Codificação dos estados
      2. Tabelas de transição de estados
      3. Determinação das funções lógicas de saída e estado seguinte

7.2. Projeto FSM

- **Passo 3:** ***Determinação das funções lógicas de saída e estado seguinte***

Entradas			Saídas		
s_1	s_0	b	x	n_1	n_0
0	0	0	0	0	0
0	0	1	0	0	1
0	1	0	1	1	0
0	1	1	1	1	0
1	0	0	1	1	1
1	0	1	1	1	1
1	1	0	1	0	0
1	1	1	1	0	0

7.2. Projeto FSM

- **Passo 3:** ***Determinação das funções lógicas de saída e estado seguinte***

Entradas			Saídas		
s_1	s_0	b	x	n_1	n_0
0	0	0	0	0	0
0	0	1	0	0	1
0	1	0	1	1	0
0	1	1	1	1	0
1	0	0	1	1	1
1	0	1	1	1	1
1	1	0	1	0	0
1	1	1	1	0	0

$x = s_1 + s_0$

$n1 = s_1' \cdot s_0 + s_1 \cdot s_0'$

$n0 = s_1' \cdot s_0' \cdot b + s_1 \cdot s_0'$

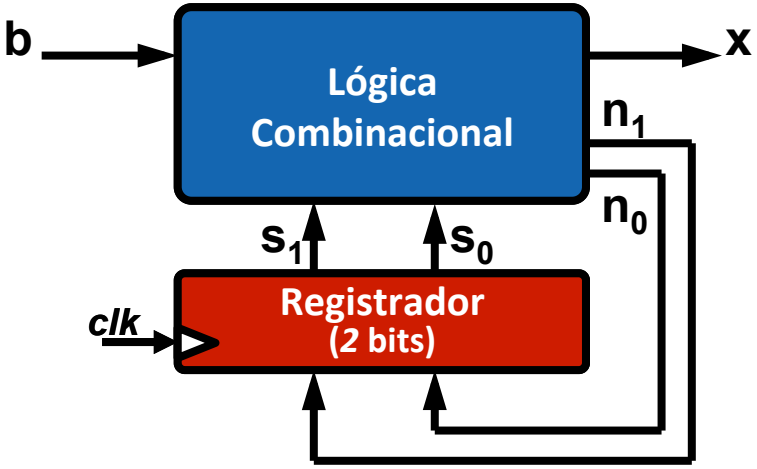
7.2. Projeto FSM

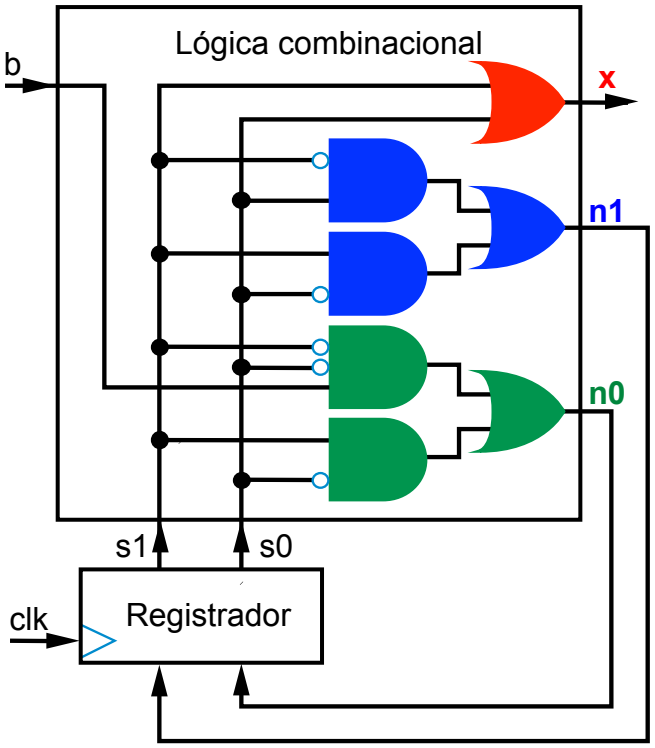
- **Passo 3:** ***Determinação das funções lógicas de saída e estado seguinte***

$x = s_1 + s_0$

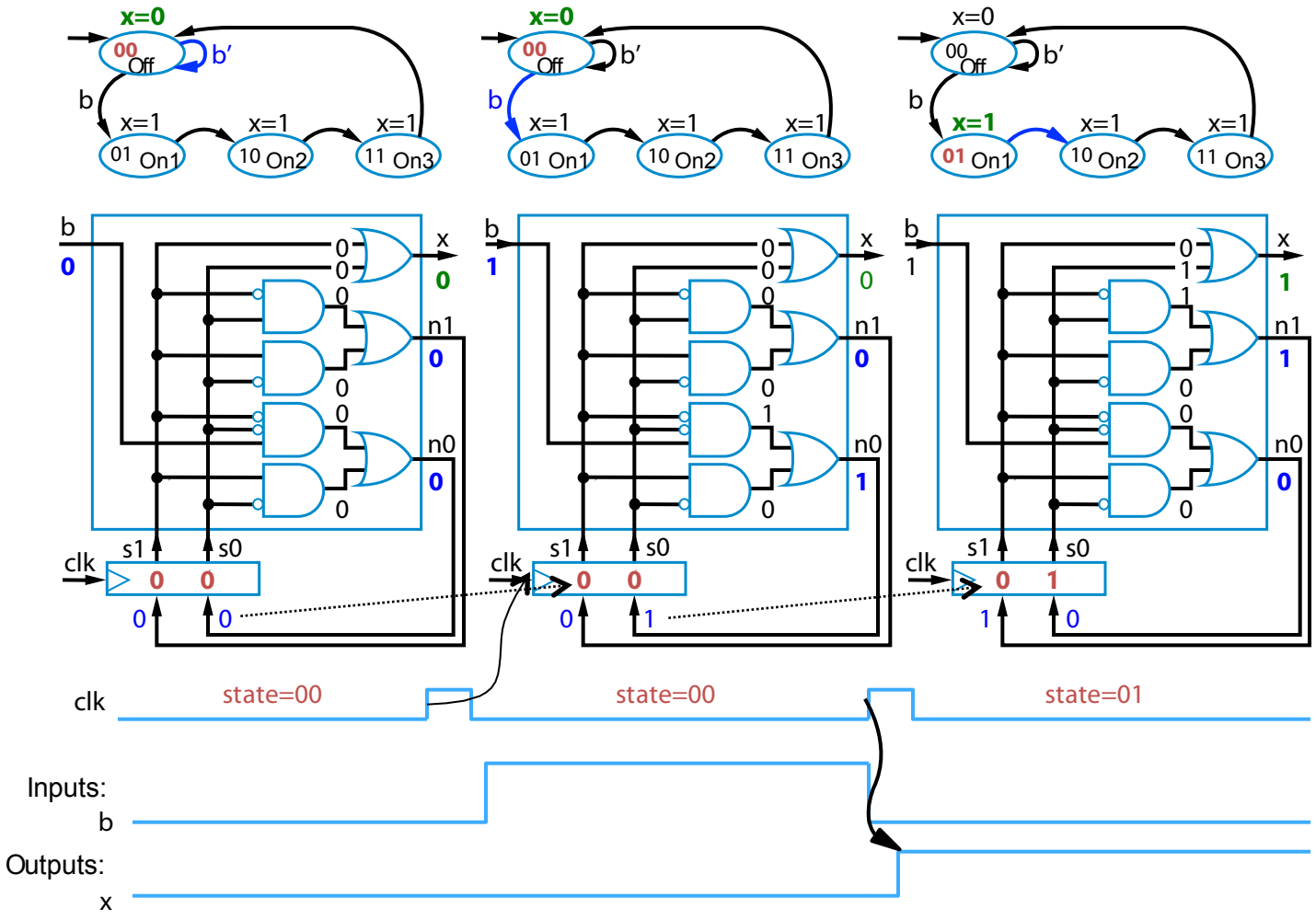
$n1 = s_1' \cdot s_0 + s_1 \cdot s_0'$

$n0 = s_1' \cdot s_0' \cdot b + s_1 \cdot s_0'$





7.2. Projeto FSM

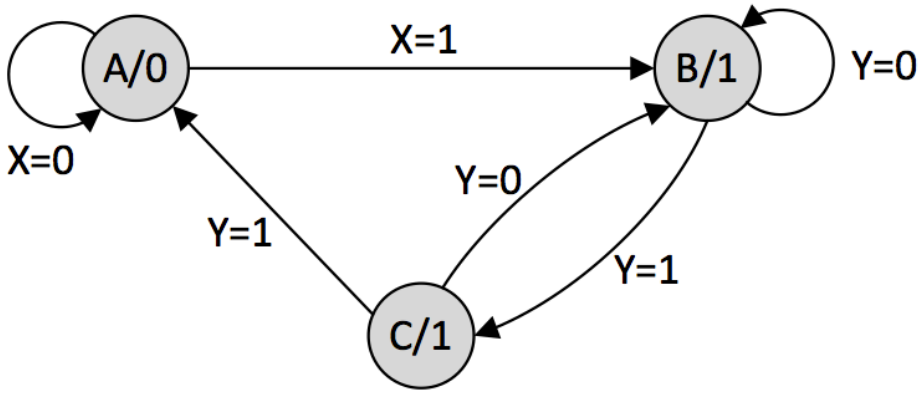


PROBLEMAS

Problema 7.3. Considere o seguinte diagrama de estados de um circuito sequencial síncrono, caracterizado por duas entradas (X,Y) e uma saída (Z):

a) Apresente, a tabela de transição de estados deste circuito. Considere a utilização de uma codificação binária e de flip-flops do tipo D.

b) Sintetize as funções lógicas correspondentes às entradas dos flip-flops e à saída do circuito.



PROBLEMAS

Problema 7.9. Considere um contador de 2 bits com contagem ascendente, pausa e indicação de máximo da contagem.

a) Faça o diagrama de estados

b) Apresente a tabela de transição de estados deste circuito. Considere a utilização de uma codificação binária e de flip-flops do tipo D.

c) Sintetize as funções lógicas correspondentes às entradas dos flip-flops e à saída do circuito.

QUESTÃO 2: Tempo 5 min



FEDERAL UNIVERSITY
OF SANTA CATARINA

EEL5105 – Circuitos e Técnicas Digitais
Aula 7

Prof. Héctor Pettenghi

hector@eel.ufsc.br
http://hectorpettenghi.paginas.ufsc.br

Material desenvolvido com apoio de arquivos de apresentação do livro de Frank Vahid

Exercícios

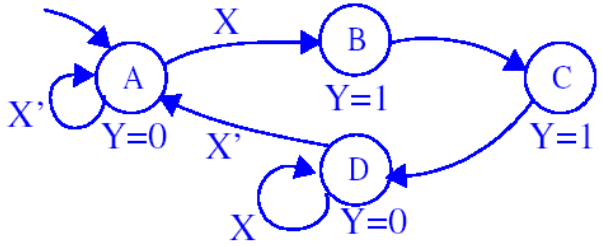
- Exercícios do Livro do **Frank Vahid**
  - **3.23** até **3.27**
  - **3.29** e **3.30**
  - **3.32** e **3.33**
  - **3.38** até **3.42**
- **A versão digital do livro do Frank Vahid está disponível no site da BU**
  - Mais especificamente em: http://150.162.4.10/pergamum/biblioteca_s/php/login_pearson.php

Exercícios

- Respostas Livro do **Frank Vahid**

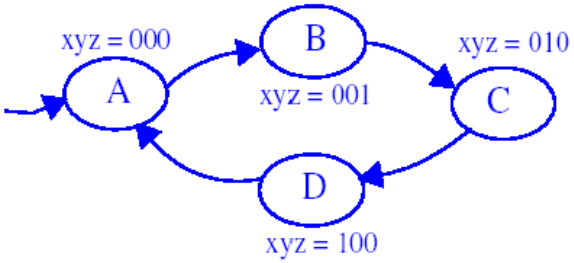
3.23

Inputs: X, *Outputs*: Y



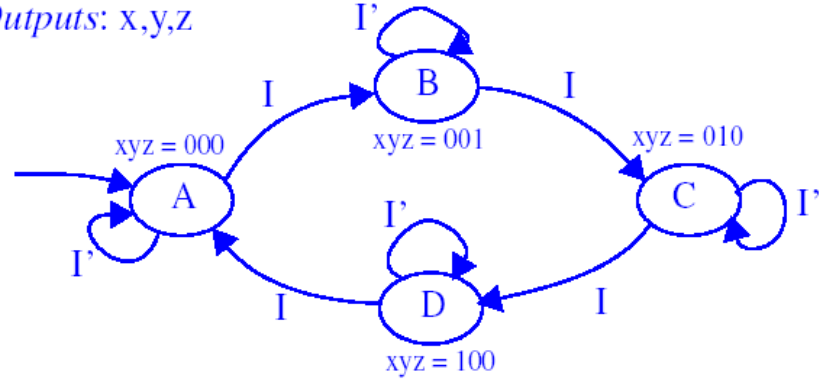
3.24

Inputs: None, *Outputs*: x,y,z



3.25

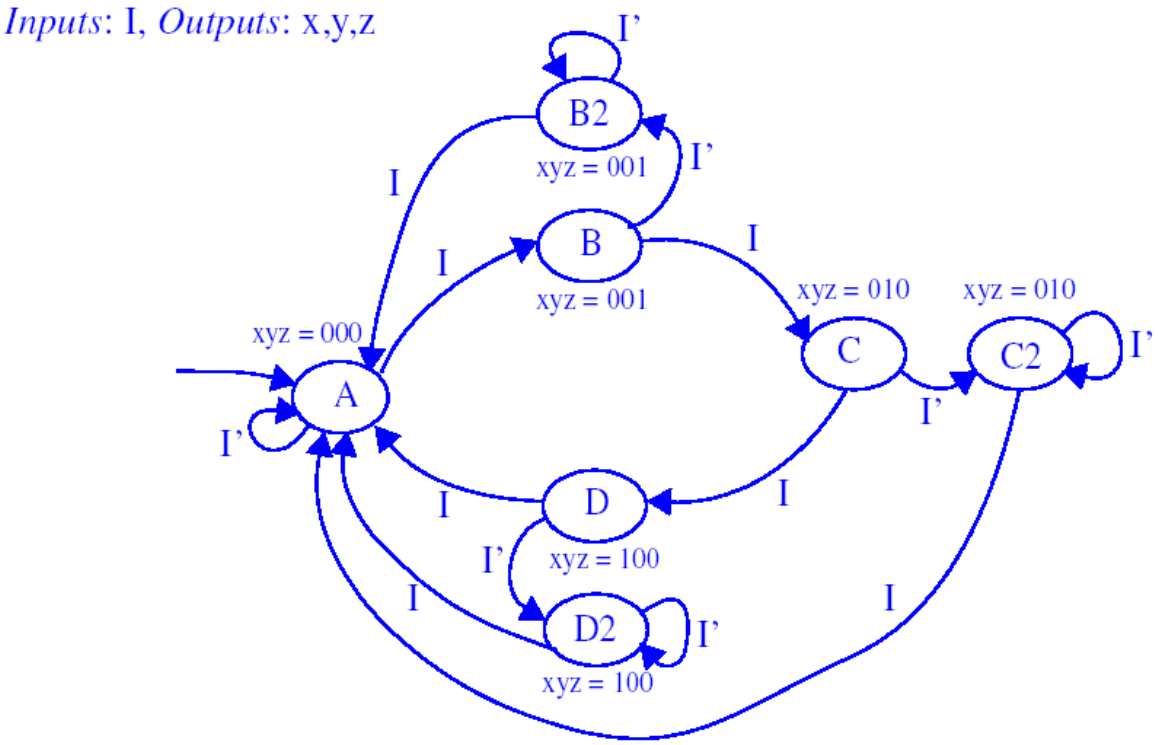
Inputs: I, *Outputs*: x,y,z



Exercícios

- Respostas Livro do **Frank Vahid**

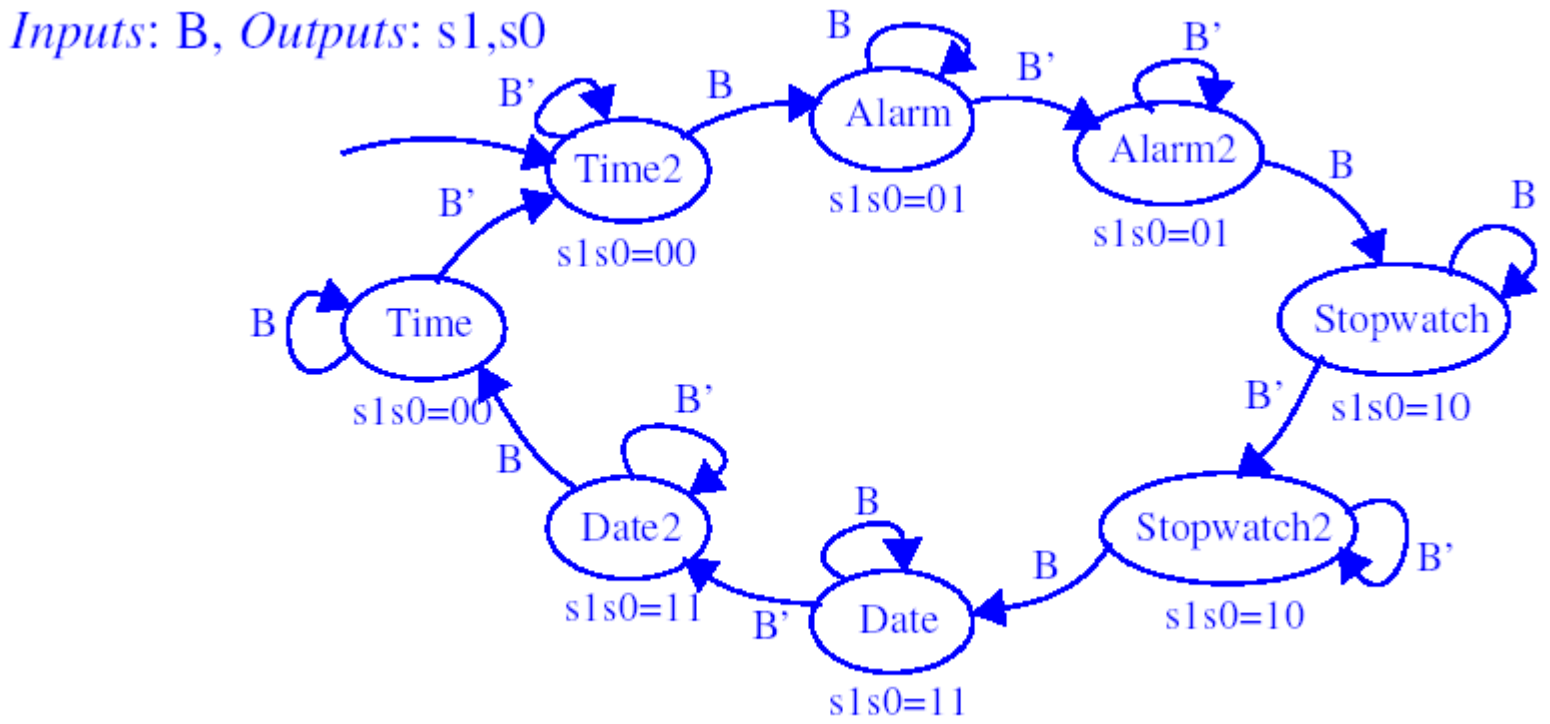
3.26



Exercícios

- Respostas Livro do **Frank Vahid**

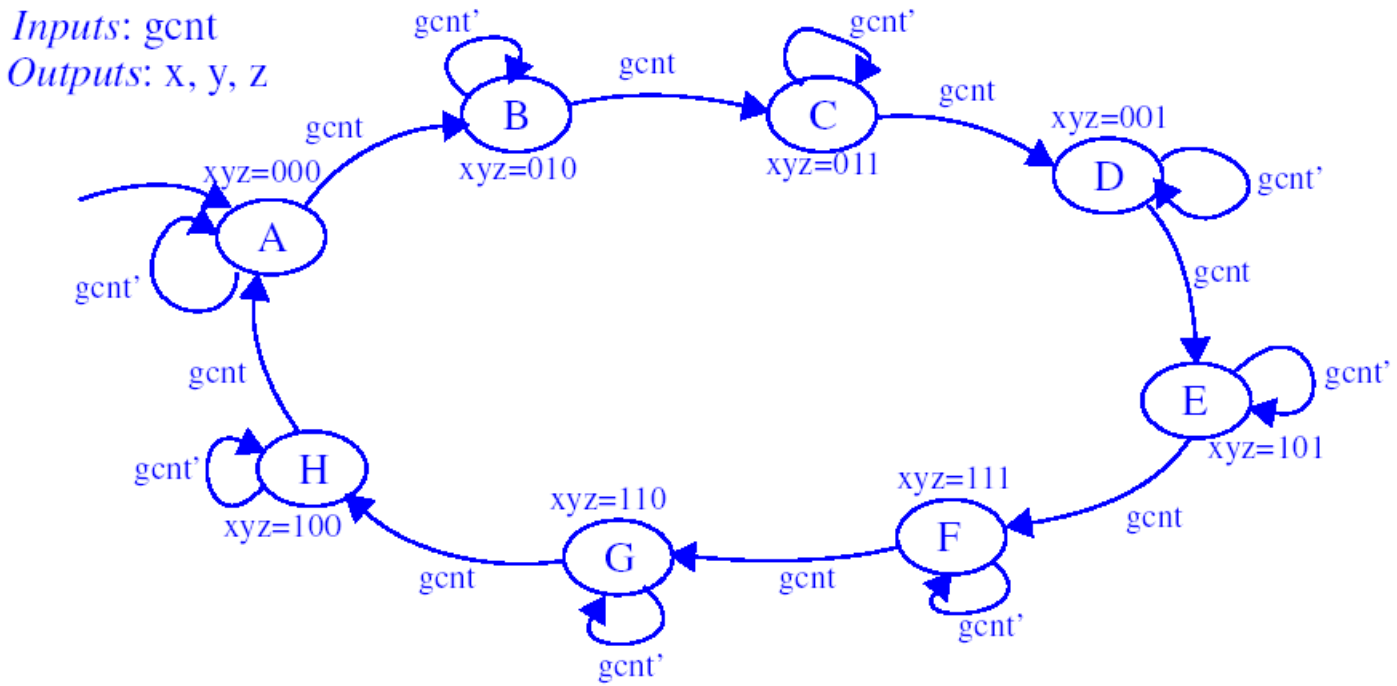
3.27



Exercícios

- Respostas Livro do **Frank Vahid**

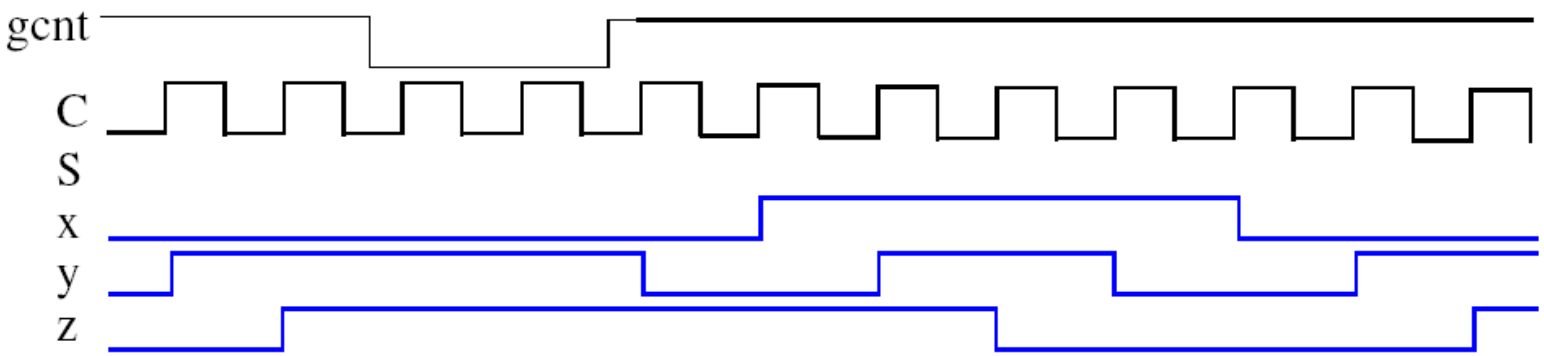
3.29



Exercícios

- Respostas Livro do **Frank Vahid**

3.30



Exercícios

- Respostas Livro do **Frank Vahid**

3.32

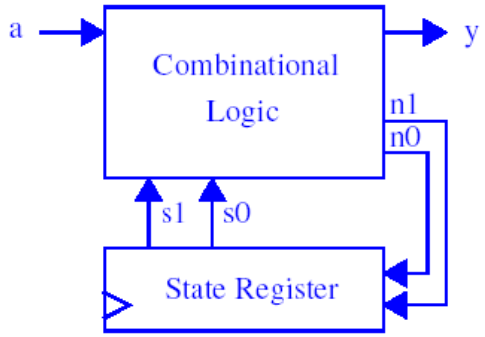
a) 2 bits
b) 3 bits
c) 4 bits
d) 5 bits
e) 10 bits

3.33

$2^{16} = 65{,}536$ possible states

Exercícios

- Resposta do **3.38**



A straightforward encoding is A=00, B=01, C=10, D=11.

Inputs			Outputs		
s1	s0	a	n1	n0	y
0	0	0	0	1	0
0	0	1	0	0	0
0	1	0	0	1	1
0	1	1	1	0	1
1	0	0	1	1	1
1	0	1	1	1	1
1	1	0	0	0	0
1	1	1	0	0	0

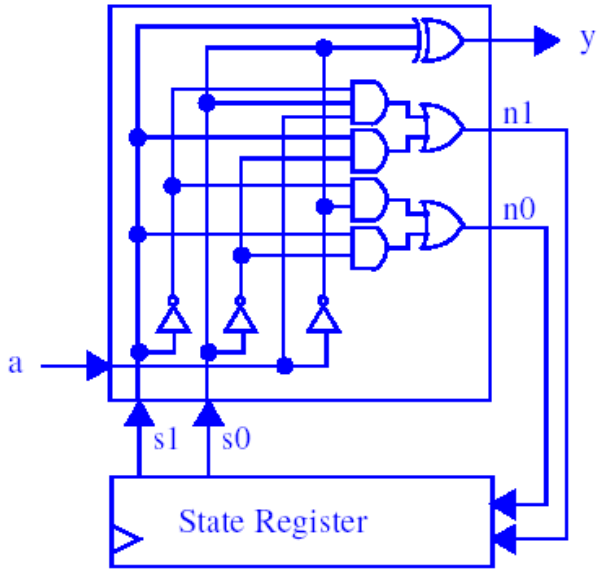
Exercícios

- Resposta do **3.38**

$n1 = s1's0a + s1s0'a' + s1s0'a = s1's0a + s1s0'$

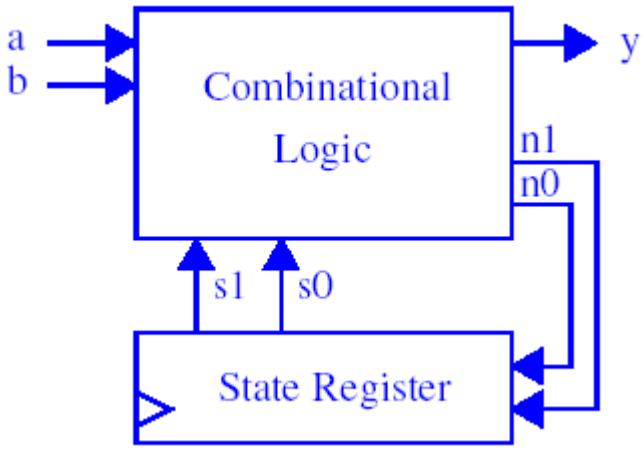
$n0 = s1's0'a' + s1's0a' + s1s0'a' + s1s0'a = s1'a' + s1s0'$

$y = s1's0a' + s1's0a + s1s0'a' + s1s0'a = s1's0 + s1s0' = s1 \text{ xor } s0$



Exercícios

- Resposta do **3.39**



Inputs				Outputs		
s1	s0	a	b	n1	n0	y
0	0	0	0	1	0	0
0	0	0	1	0	1	0
0	0	1	0	0	0	0
0	0	1	1	0	0	0
0	1	0	0	0	1	1
0	1	0	1	0	1	1
0	1	1	0	1	0	1
0	1	1	1	1	0	1
1	0	0	0	1	0	1
1	0	0	1	1	1	1
1	0	1	0	1	0	1
1	0	1	1	1	1	1
1	1	0	0	0	0	0
1	1	0	1	0	0	0
1	1	1	0	0	0	0
1	1	1	1	0	0	0

A straightforward encoding is A=00, B=01, C=10, D=11.

Exercícios

- Resposta do **3.39**

Step 5 - Implement the combinational logic

n1 = s1's0'a'b' + s1's0a + s1s0'

n0 = s1's0'a'b + s1's0a' + s1s0'b

y = s1's0 + s1s0'

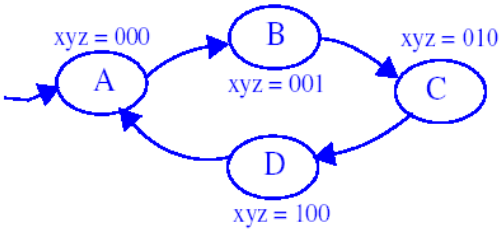
Note: The above equations can be minimized further.

Exercícios

- Resposta do **3.40**

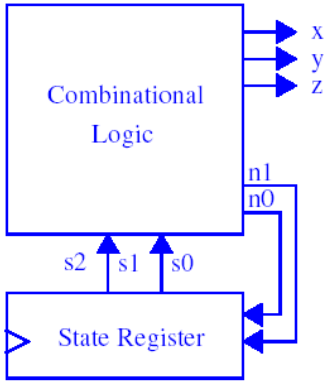
Step 1 - Capture the FSM

Inputs: None, *Outputs*: x,y,z



The FSM was created during Exercise 3.24.

Step 2 - Create the architecture



Step 3 - Encode the states

A straightforward encoding is A=00, B=01, C=10, D=11.

Step 4 - Create the state table

Inputs		Outputs				
s1	s0	n1	n0	x	y	z
0	0	0	1	0	0	0
0	1	1	0	0	0	1
1	0	1	1	0	1	0
1	1	0	0	1	0	0

Step 5 - Implement the combinational logic

n1 = s1's0 + s1s0' = s1 XOR s0

n0 = s1's0' + s1s0' = s0'

x = s1s0

y = s1s0'

z = s1's0